

DOCUMENT TECHNIQUE

FNAF 1 — Java Remake

Architecture · Classes · Base de données · Logique de jeu

Projet	Five nights at Java's
Langage	Java (JDK 17+)
Interface graphique	Swing
Base de données	SQLite / MySQL via JDBC
Version du document	v1.0 — 2024/2025
Auteur(s)	Arnaud GUILLERMIN, Théophile MARTIN, Lilian LABETOULLE

1. Introduction

1.1 Contexte et objectif :

Ce document technique décrit l'architecture, les choix d'implémentation, les classes, les fonctions et la base de données de notre projet.

L'objectif est de recréer le jeu Five Nights at Freddy's en proposant certaines des mécaniques : surveillance par caméras, fermeture de portes, et intelligence artificielle des animatroniques.

1.2 Technologies utilisées :

- **Langage principal** : Java JDK 17+
- **Interface graphique** : Swing
- **Accès à la base de données** : JDBC (Java Database Connectivity)
- **Système de base de données** : SQLite
- **Gestion audio** : javax.sound.sampled et JavaFX Media
- **Ressources (images, sons)** : Chargement depuis la base de données

1.3 Organisation du projet :

L'arborescence recommandée du projet est la suivante :

```
src/  
├─ Main/           → Point d'entrée (Main.java)  
├─ Joueur/         → Joueur et touches  
├─ Animatronique/  → Animatroniques  
├─ Caméra/         → Images des caméras  
├─ Porte/          → Ouverture et fermeture des portes  
├─ Lumière/        → Allumage et extinction des lumières  
├─ Piece/          → Définition des pièces  
├─ Bouton_lumiere/ → Action de changer l'état des lumières  
└─ Bouton_porte/   → Action de changer l'état des portes
```

2. Base de données

La base de données contient l'intégralité des sons et des images du jeu. Elle est donc appelée dans plusieurs classes, qui ont besoin de récupérer le contenu de la base.

Tables :

Table : Image	Type	Description
id	INTEGER PK	Identifiant unique des images
nom	VARCHAR()	Nom de l'image
adresse	VARCHAR()	Adresse de l'image

Table : Son	Type	Description
id	INTEGER PK	Identifiant des sons
nom	VARCHAR()	Nom du son
adresse	VARCHAR()	Adresse du son

4. Classes principales

4.1 Vue d'ensemble

Chaque classe ci-dessous est décrite avec son rôle, ses attribut et ses méthodes

Classe : Main	
Rôle	Point d'entrée du programme. Initialise la fenêtre et lance la boucle de jeu.
Attributs	Aucun
Méthodes	main(String[] args), setPopulation(), newPopulation(), setTime(), chrono(), move(), redemarrer_pos_anim(), get_adresse_image(), get_adresse_son(), Jumpscare(), porteGauche(), porteDroite(), porteVide()

Classe : Joueur	
Rôle	Crée l'entité du joueur, et initialise les touches de l'utilisateur
Attributs	Lumiere_gauche, Lumiere_droite, Bouton_lumiere_gauche, Bouton_lumiere_droite, Porte_gauche, Porte_droite, Bouton_porte_gauche, Boutone_porte_droite, num_fenetre
Méthodes	Joueur(), ActionTouches, kayPressed

Classe : Animatronique	
Rôle	Crée les animatroniques et leur système de mouvement
Attributs	id_espece, num_position, time, move, tour
Méthodes	setChrono(), decomppte()

Classe : Caméra	
Rôle	Crée les caméras, et retourne leurs images selon leur situation
Attributs	etat, num_piece, lumiere_gauche, lumiere_droite
Méthodes	allumerCameras(), fermerCameras(), get_adresse_image(), get_adresse_son(), afficherCameras()

Classe : Porte	
Rôle	Crée les portes et permet de les ouvrir et fermer
Attributs	<code>num_porte</code> , <code>etat</code>
Méthodes	<code>get_adresse_son()</code> , <code>ouvrirPorte()</code> , <code>fermerPorte()</code>

Classe : Lumière	
Rôle	Crée les lumières, permet de les allumer et de les éteindre et détecte si un animatronique est à la porte
Attributs	<code>num_lumiere</code> , <code>etat</code> , <code>animatroniques</code>
Méthodes	<code>Get_adresse_son()</code> , <code>allumerLumiere()</code> , <code>eteindreLumiere()</code> , <code>AnimPorte()</code>

Classe : Piece	
Rôle	Crée les pièces et y ajoute les animatroniques
Attributs	<code>id_piece</code> , <code>place_max</code> , <code>population</code>
Méthodes	<code>ajouterAnim()</code> , <code>supprimerAnim()</code>

Classe : Bouton_lumiere	
Rôle	Crée les boutons des lumières permettant de changer leur état
Attributs	<code>num_bouton_lumiere</code> , <code>lumiere</code>
Méthodes	<code>appuyerBoutonLumiere()</code>

Classe : Bouton_porte	
Rôle	Crée les boutons des portes permettant de changer leur état
Attributs	<code>num_bouton_porte</code> , <code>porte</code>
Méthodes	<code>appuyerBoutonPorte()</code>

5. Logique de jeu

5.1 Boucle principale

La boucle de jeu commence par créer toutes les pièces, entités, portes et lumières. Ensuite, 2 threads sont utilisés afin que les fonctions des différentes classes puissent fonctionner en continu.

5.2 Intelligence artificielle des animatroniques

Chaque animatronique se déplace au bout d'un certain temps défini au début du programme,

avec un ordre précis : Bonnie, Chica et enfin Freddy. Foxy lui est indépendant des autres.

- Freddy : se déplace du côté droit de la carte.
- Bonnie : attaque par la porte gauche.
- Chica : attaque par la porte droite.
- Foxy : sprint si non surveillé pendant trop longtemps.

5.3 Gestion des portes et lumières

Le joueur les contrôle via des boutons dans l'interface. Si une porte est fermée au moment où un animatronique arrive, il repart.

5.4 Conditions de fin de partie

- Game over : un animatronique entre dans le bureau (porte non fermée) → jumpscare

6. Gestion des ressources multimédia

6.1 Images

Les images sont au format PNG. Elles sont chargées lorsque nécessaires via les différentes fonctions les appelant et sont stockées dans la base de données.

6.2 Sons

Les sons sont au format WAV (compatibilité native Java sans librairie tierce). Chaque son est stocké dans la base de données et appelé lorsque nécessaire.

8. Difficultés rencontrées et solutions

Problème rencontré	Solution apportée
<i>Non affichage des images</i>	Modification du code
<i>Non chargement des sons</i>	Compression des sons dans VLC Media Player